

Stored Procedures / Functions

JackPots World Tours & Travels — Phase 1 PostgreSQL Backend — Historical Record

Step 39.1: Create Booking Request

Definition

```
CREATE OR REPLACE FUNCTION create_booking_request(
    p_merchant_id INT,
    p_user_id INT,
    p_travel_type VARCHAR,
    p_service_name VARCHAR,
    p_from_location VARCHAR,
    p_to_location VARCHAR,
    p_travel_date DATE,
    p_return_date DATE,
    p_passengers INT,
    p_remarks TEXT
)
RETURNS INT
LANGUAGE plpgsql
AS
$$
DECLARE
    new_request_id INT;
BEGIN
    INSERT INTO service_requests
    (
        merchant_id,
        user_id,
        request_type,
        booking_reference,
        travel_type,
        service_name,
        from_location,
        to_location,
        travel_date,
        return_date,
        number_of_passengers,
        remarks
    )
    VALUES
    (
        p_merchant_id,
        p_user_id,
        'BOOKING',
        'JPW-' || TO_CHAR(NOW(), 'YYYYMMDDHH24MISS'),
        p_travel_type,
        p_service_name,
        p_from_location,
        p_to_location,
        p_travel_date,
        p_return_date,
        p_passengers,
        p_remarks
    )
    RETURNING request_id
```

```

        INTO new_request_id;

        RETURN new_request_id;
END;
$$;

```

Test

```

SELECT create_booking_request(
    1,
    1,
    'FLIGHT',
    'Hyderabad to Dubai',
    'Hyderabad',
    'Dubai',
    '2026-08-20',
    '2026-08-28',
    2,
    'Window seat'
);

```

Step 39.2: Approve Booking

Definition

```

CREATE OR REPLACE FUNCTION approve_booking(
    p_request_id INT,
    p_admin_id INT
)
RETURNS VOID
LANGUAGE plpgsql
AS
$$
BEGIN
    UPDATE service_requests
    SET
        status='APPROVED',
        approved_by=p_admin_id,
        completed_at=CURRENT_TIMESTAMP
    WHERE request_id=p_request_id;
END;
$$;

```

Test

```

SELECT approve_booking(1,1);

```

Step 39.3: Cancel Booking

Definition

```

CREATE OR REPLACE FUNCTION cancel_booking(
    p_request_id INT,
    p_reason TEXT
)
RETURNS VOID
LANGUAGE plpgsql

```

```

AS
$$
BEGIN
    UPDATE service_requests
    SET
        status='CANCELLED',
        remarks = remarks || E'\nCancellation Reason: ' || p_reason
    WHERE request_id=p_request_id;
END;
$$;

```

Test

```

SELECT cancel_booking(
    1,
    'Customer requested cancellation.'
);

```

Step 39.4: Process Refund

Definition

```

CREATE OR REPLACE FUNCTION process_refund(
    p_payment_id INT,
    p_amount DECIMAL
)
RETURNS VOID
LANGUAGE plpgsql
AS
$$
BEGIN
    UPDATE payments
    SET
        refund_amount=p_amount,
        payment_status='REFUNDED'
    WHERE payment_id=p_payment_id;
END;
$$;

```

Test

```

SELECT process_refund(
    1,
    5000
);

```

Step 39.5: Add Passenger

Definition

```

CREATE OR REPLACE FUNCTION add_passenger(
    p_request_id INT,
    p_merchant_id INT,
    p_first_name VARCHAR,
    p_last_name VARCHAR,
    p_gender VARCHAR,
    p_passport VARCHAR

```

```

)
RETURNS VOID
LANGUAGE plpgsql
AS
$$
BEGIN
    INSERT INTO passenger_data
    (
        request_id,
        merchant_id,
        first_name,
        last_name,
        gender,
        passport_number
    )
    VALUES
    (
        p_request_id,
        p_merchant_id,
        p_first_name,
        p_last_name,
        p_gender,
        p_passport
    );
END;
$$;

```

Test

```

SELECT add_passenger(
    1,
    1,
    'Ramesh',
    'Kumar',
    'Male',
    'P99887766'
);

```

Step 39.6: Record Payment

Definition

```

CREATE OR REPLACE FUNCTION record_payment(
    p_merchant_id INT,
    p_request_id INT,
    p_amount DECIMAL,
    p_method VARCHAR,
    p_transaction_id VARCHAR
)
RETURNS VOID
LANGUAGE plpgsql
AS
$$
BEGIN
    INSERT INTO payments
    (
        merchant_id,
        request_id,
        amount,

```

```

        payment_method,
        transaction_id,
        payment_status,
        payment_date
    )
VALUES
(
    p_merchant_id,
    p_request_id,
    p_amount,
    p_method,
    p_transaction_id,
    'SUCCESS',
    CURRENT_TIMESTAMP
);
END;
$$;

```

Test

```

SELECT record_payment(
    1,
    1,
    25000,
    'UPI',
    'TXN123456789'
);

```

Step 39.7: Merchant Dashboard Statistics

Definition

```

CREATE OR REPLACE FUNCTION merchant_dashboard(
    p_merchant_id INT
)
RETURNS TABLE
(
    total_requests BIGINT,
    total_payments BIGINT,
    total_revenue NUMERIC
)
LANGUAGE plpgsql
AS
$$
BEGIN
    RETURN QUERY
    SELECT
        COUNT(DISTINCT sr.request_id),
        COUNT(DISTINCT p.payment_id),
        COALESCE(SUM(p.amount),0)
    FROM service_requests sr
    LEFT JOIN payments p
        ON sr.request_id=p.request_id
    WHERE sr.merchant_id=p_merchant_id;
END;
$$;

```

Test

```
SELECT * FROM merchant_dashboard(1);
```

Summary

All seven stored procedures are callable directly from the FastAPI application via SQLAlchemy or asyncpg:

The diagram illustrates the role of the `merchant_dashboard()` function within a data science project. On the left, a vertical column of eight dark gray circles represents the project's components. The bottom-most circle is highlighted with a white border. A horizontal line extends from this circle to the right, where the text `merchant_dashboard()` is displayed. This text is positioned at the top of a large, light gray rectangular area that occupies the right half of the diagram, representing the dashboard output.